

VIRTUAL MOUSE USING HAND GESTURES RECOGNITION SYSTEM

A PROJECT REPORT

Submitted by

L.NAVEENBABU(810022205106)

S.PRAVEEN KUMAR(810022205088)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY



**UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS,
TIRUCHIRAPPALLI**

ANNA UNIVERSITY: CHENNAI 600 025

MAY 2026

**UNIVERSITY COLLEGE OF ENGINEERING,
BIT CAMPUS,
TIRUCHIRAPPALLI-620 024**

BONAFIDE CERTIFICATE

Certified that this project report “**VIRTUAL MOUSE USING HAND GESTURES RECOGNITION SYSTEM**” is the Bonafide work of **L.NAVEENBABU(810022205106)** and **S.PRAVEEN KUMAR (810022205088)** who carried out the project work under my supervision.

SIGNATURE

Dr.BLESSY SELVAM,

Guest Faculty,

Department of Computer Science and
Engineering,

University College of Engineering,

Anna University-BIT Campus,

Tiruchirappalli-620 024.

SIGNATURE

Dr. G. ANNAPOORANI,

HEAD OF THE DEPARTMENT,

Assistant Professor(SI Gr),

Department of CSE/IT,

University College of Engineering,

Anna University-BIT Campus,

Tiruchirappalli-620 024.

Submitted for VIVA VOICE examination held at University College of
Engineering on_____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare the project work entitled “**VIRTUAL MOUSE USING HAND GESTURES RECOGNITION SYSTEM**” is submitted in partial fulfillment of the requirements for the award of the degree in B.TECH. (Information Technology), in University College of Engineering, BIT Campus, Anna University, Tiruchirappalli. This is a record of our own work carried out during the academic year 2025 – 2026 under the supervision and guidance of **Dr.BLESSY SELVAM**, Guest Faculty, Department of Computer Science and Engineering, University College of Engineering, BIT Campus, Anna University, Tiruchirappalli. The extent and source of information are derived from the existing literature and have been indicated through the dissertation at the appropriate places. The matter embodied in this work is original and has not been submitted for the award of any other degree at this or any other university.

SIGNATURE OF THE CANDIDATE

NAVEENBABU L

(810022205106)

SIGNATURE OF THE CANDIDATE

PRAVEEN KUMAR S

(810022205088)

I certify that the declaration made by the candidates are true.

SIGNATURE OF THE GUIDE

Dr.BLESSY SELVAM,

Guest Faculty,

Department of CSE,

University College of Engineering,

BIT Campus, Anna University,

Tiruchirappalli – 620024.

ACKNOWLEDGEMENT

We are deeply grateful to our Honorable Dean, **Dr. T. Senthilkumar, Professor**, for providing all the necessary resources and support that facilitated the smooth progress and completion of our Final Year Project.

We extend our heartfelt thanks to, **Dr. G. Annapoorani, Assistant Professor and Head of the Department of Computer Science and Engineering/ Information Technology**, for her expert guidance, thoughtful suggestions, and continued encouragement, which were instrumental in the successful completion of this work.

We also sincerely thank our Project Coordinator, **Mrs. C. Kanimozhi, Associate Professor, Department of Information Technology**, for her consistent support, motivation, and coordination efforts throughout this project.

A special note of appreciation goes to, **Dr. Blessy Selvam, Guest Faculty, Department of Computer Science and Engineering**, for her steadfast support, invaluable feedback, and dedicated mentoring throughout every phase of this project.

We also extend our sincere gratitude to the Teaching and Non-Teaching Staff and Lab Assistants of the Department of Computer Science and Engineering for their timely technical support and guidance throughout the project. Above all, we are deeply grateful to our beloved parents and siblings for their unconditional love, constant prayers, and endless encouragement, which remained our greatest strength throughout this journey.

Finally, we warmly thank our friends for their support, motivation, and companionship, which made this journey truly memorable and enjoyable.

ABSTRACT

This project presents GestureOS, a real-time hand gesture recognition system that enables users to control their computer without touching the keyboard or mouse. The rapid advancement of computer vision and machine learning has enabled more natural human-computer interaction. Traditional input devices like the mouse and keyboard impose physical and accessibility constraints in certain environments and for some users. GestureOS addresses these limitations using Google's MediaPipe Hands framework combined with OpenCV for real-time hand tracking through a standard webcam. The system detects twenty one three-dimensional hand landmarks per frame and applies a three-point finger classification algorithm to identify fourteen unique hand gestures, each mapped to specific system actions such as cursor movement, mouse clicks, scrolling, zooming, volume and brightness control, screenshot capture, and system lock. A key contribution of this project is camera-based depth estimation without requiring a depth sensor. By calculating the Euclidean distance between the wrist and middle finger base landmarks, the system determines hand distance from the camera and triggers depth-based gestures, eliminating the need for additional hardware. The system also includes a Flask and Socket.IO-based web dashboard that allows users to control the gesture engine in real time. It provides live feedback such as detected gestures, session duration, hand detection status, and engine state. This project demonstrates the integration of computer vision, machine learning, operating system control, and web development into a unified and practical system.

TABLE OF CONTENTS

S.NO	TITTLE	PAGE NO
	ABSTRACT	v
	LIST OF FIGURES	xii
	LIST OF TABLES	xiii
	LIST OF ABBREVIATIONS	xiV
1	INTRODUCTION	
	1.1 Overview of the Project	1
	1.2 Problem Statement	2
	1.3 Objectives	3
	1.4 Scope of the Project	3
	1.5 Motivation	4
2	LITERATURE SURVEY	5
3	EXISTING WORK	
	3.1 Introduction to Existing Systems	8
	3.2 Hardware-Based Gesture Recognition Systems	8
	3.2.1 Data Glove-Based Systems	8
	3.2.2 Depth Sensor and Infrared-Based Systems	8
	3.3 Vision-Based Gesture Recognition Systems	8
	3.3.1 RGB Camera-Based Approaches	8
	3.3.2 Deep Learning-Based Gesture Recognition	9
	3.4 Hand Tracking Frameworks	9
	3.4.1 MediaPipe Hands	9
	3.4.2 Other Hand Tracking Models	9

3.5 Image Processing Techniques	9
3.5.1 Role of OpenCV	9
3.5.2 Preprocessing and Feature Extraction	10
3.6 Gesture Classification Techniques	10
3.6.1 Machine Learning-Based Methods	10
3.6.2 Rule-Based Classification Approaches	10
3.7 Depth Estimation Methods	10
3.7.1 Sensor-Based Depth Detection	10
3.7.2 Camera-Based Depth Approximation	10
3.8 Applications of Gesture Recognition Systems	11
3.8.1 Human-Computer Interaction	11
3.8.2 Accessibility and Assistive Technologies	11
3.9 Limitations of Existing Systems	11
3.10 Research Gap and Motivation	11

4 PROPOSED SYSTEM

4.1 Introduction to Proposed System	12
4.2 System Architecture	12
4.2.1 Layered Architecture Overview	13
4.2.2 Presentation Layer (User Interface)	13
4.2.3 Application Layer (Server Logic)	13
4.2.4 Processing Layer (Gesture Intelligence)	13
4.2.5 Hardware Layer (Physical Components)	13
4.3 Workflow of GestureOS System	14
4.3.1 Input Capture using Webcam	14

4.3.2 Frame Processing using OpenCV	14
4.3.3 Hand Landmark Detection	14
4.3.4 Finger State Classification	15
4.3.5 Distance Calculation (Euclidean Method)	15
4.3.6 Depth Estimation (Hand Size Proxy)	15
4.3.7 Gesture Matching Logic	15
4.3.8 System Command Execution	15
4.3.9 Real-Time Dashboard Update	15
4.4 Web Dashboard Design	15
4.4.1 index.html Interface	15
4.4.2 Live Status Panel	16
4.4.3 Gesture Feedback Display	16
4.4.4 Session Monitoring	16
4.5 OS Integration	16
4.5.1 PyAutoGUI for Mouse/Keyboard Control	16
4.5.2 Pycaw for Audio Control	16
4.5.3 System Brightness Control APIs	16
4.6 Advantages of Proposed System	16
4.7 Applications of the Proposed System	16
4.8 Limitations	18
4.9 Use Case Diagram	19
4.10 Class Diagram	20
4.11 Activity Diagram	22

5	SYSTEM REQUIREMENTS	
	5.1 Hardware Requirements	24
	5.2 Software Requirements	25
	5.2.1 Libraries and Frameworks	26
	5.2.2 Other Requirements	27
6	SYSTEM IMPLEMENTATION	
	6.1 Introduction to System Implementation	28
	6.2 Development Environment Setup	28
	6.3 System Architecture Implementation	28
	6.4 Video Capture and Processing	28
	6.5 Hand Detection and Landmark Extraction	29
	6.6 Gesture Recognition and Classification	29
	6.7 Depth Estimation Technique	29
	6.8 System Control and OS Integration	29
	6.9 Web Dashboard Implementation	29
	6.10 Performance Optimization	30
	6.11 Testing and Validation	30
	6.12 Final System Integration and Execution	30
7	EXPERIMENTAL RESULT AND DISCUSSION	
	7.1 Introduction to Experimental Setup	31
	7.2 Testing Environment	31
	7.2.1 Hardware Configuration	31
	7.2.2 Software Environment	31
	7.2.3 Lighting and Background Conditions	31

7.3 Performance Evaluation Metrics	31
7.3.1 Accuracy of Gesture Recognition	31
7.3.2 Response Time (Latency)	32
7.3.3 Frame Rate (FPS)	32
7.4 Experimental Results	32
7.4.1 Gesture Detection Results (14 Gestures)	32
7.4.2 System Functionality Testing	32
7.4.3 Depth Estimation Performance	32
7.5 Result Analysis	32
7.5.1 Accuracy under Different Conditions	32
7.5.2 Effect of Lighting and Background	33
7.5.3 Performance with Different Users	33
7.6 Comparison with Existing Systems	33
7.7 Advantages Observed	33
7.8 Limitations Identified	33
7.9 Discussion	33
7.10 Summary of Results	33

8 CONCLUSION AND FUTURE SCOPE

8.1 Conclusion	34
8.2 Future Enhancements	35
8.2.1 Custom Gesture Training	35
8.2.2 Multi-Hand Support	35
8.2.3 Mobile App Integration	35
8.2.4 Cross-Platform Support (Linux / Mac)	35

	8.2.5 Accessibility Mode for Differently-Abled Users	35
	8.2.6 Real-Time Performance Analytics	36
	8.2.7 Gaming and Application Profiles	36
	8.2.8 Voice Gesture Fusion	36
9	ANNEXURE I	37
	ANNEXURE II	43
10	REFERENCES	46

LIST OF FIGURES

S.No	Title	Page
1.1	GestureOS processes each webcam frame through a 6-stage pipeline from hand gesture to OS action	2
4.1	Four layer architecture separating presentation, application, processing and hardware concerns	12
4.2	Complete data flow from hand gesture input to OS output and web dashboard notification	14
4.3	Use case diagram for virtual mouse using hand recognition system	19
4.4	Class diagram for virtual mouse using hand recognition system	20
4.5	Activity diagram for virtual mouse using hand recognition system	22
9.1	Landing page of the Virtual Mouse using Hand Gestures Recognition System	43
9.2	Workflow page of the Proposed System	43
9.3	Gesture references to control the cursor and some functions	44
9.4	page of the virtual mouse engine start button	44
9.5	Tech stack of the proposed system	45
9.6	Quick setup page for the proposed system	45

LIST OF TABLES

S.No	Table Name	Page No
5.1	Hardware Requirements of the Proposed System	24
5.2	Software Requirements of the Proposed System	25
5.3	Libraries and Frameworks for the Proposed System	26
5.4	Other Requirements for the Proposed System	27

LIST OF ABBREVIATION

Abbreviation	Full Form
AI	– Artificial Intelligence
API	– Application Programming Interface
BGR	– Blue Green Red (OpenCV colour format)
CNN	– Convolutional Neural Network
CPU	– Central Processing Unit
CSS	– Cascading Style Sheets
CSV	– Comma Separated Values
CV	– Computer Vision
DFD	– Data Flow Diagram
FPS	– Frames Per Second
GPU	– Graphics Processing Unit
HTML	– HyperText Markup Language
HSV	– Hue Saturation Value (colour space)
HTTP	– HyperText Transfer Protocol
IDE	– Integrated Development Environment
IP	– Internet Protocol
IT	– Information Technology
JS	– JavaScript
JSON	– JavaScript Object Notation
KNN	– K-Nearest Neighbour
LAN	– Local Area Network
MCP	– Metacarpophalangeal (finger joint)
ML	– Machine Learning
MVC	– Model View Controller

PIP	– Proximal Interphalangeal (finger joint)
OS	– Operating System
PNG	– Portable Network Graphics
RGB	– Red Green Blue (colour format)
SBC	– Screen Brightness Control (library)
SDK	– Software Development Kit
RAM	– Random Access Memory
SQL	– Structured Query Language
REST	– Representational State Transfer
SVM	– Support Vector Machine
URL	– Uniform Resource Locator
USB	– Universal Serial Bus
YCbCr	– Luminance and Chrominance (colour space)
WSGI	– Web Server Gateway Interface
XNNPACK	– Accelerated Neural Network Pack (TensorFlow Lite delegate)

CHAPTER 1

INTRODUCTION

1.1 Overview of the Project

GestureOS is a hand gesture-based computer control system that replaces traditional input devices with natural hand movements captured through a standard webcam. The name GestureOS reflects the project's goal of integrating gesture recognition directly with operating system controls, going beyond simple cursor movement to provide comprehensive system management through hand gestures. The system operates by continuously capturing video frames from the webcam using OpenCV and processing each frame through Google's MediaPipe Hands model, which detects and tracks 21 key points on the human hand in real time. These key points, known as landmarks, represent the joints and tips of each finger as well as the wrist. By analysing the relative positions of these landmarks, the system determines which fingers are extended and the depth of the hand relative to the camera.

Using a strict three-point finger detection algorithm, the system classifies the current hand pose into one of 14 predefined gesture categories. Each gesture category is mapped to a specific computer action. Mouse actions such as cursor movement, single click, double click and right click are executed through the PyAutoGUI library. System controls such as volume adjustment, brightness control, screenshot capture and system lock are implemented through specialised libraries and operating system commands.

The project also includes a web-based control dashboard built with Flask and Socket.IO. This dashboard runs on a local Flask server and is accessible through any browser on the same machine. It provides a toggle button to start and stop the gesture engine and displays live status information including the detected gesture name, session timer, camera status and engine state. The dashboard communicates with the Python

backend through a WebSocket connection, enabling real-time bidirectional communication between the browser and the gesture engine.

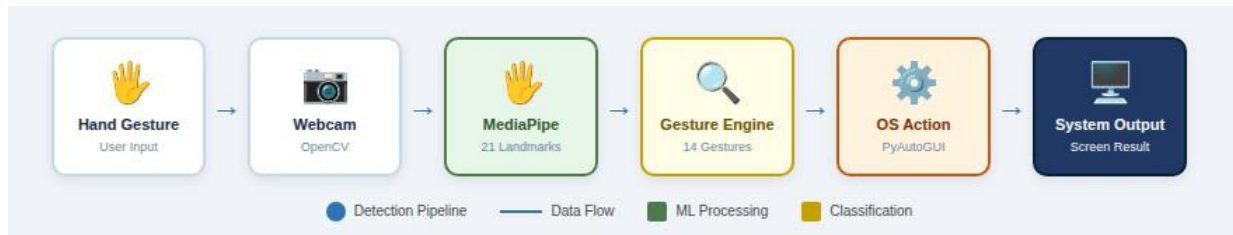


Figure 1.1: GestureOS processes each webcam frame through a 6-stage pipeline from hand gesture to OS action

1.2 Problem Statement

In the current technological landscape, the mouse and keyboard remain the dominant means of interacting with personal computers. While these devices are effective for general use, they present significant limitations in several contexts.

In healthcare environments such as hospitals, operation theatres and laboratories, touching shared input devices poses a risk of cross-contamination and infection transmission. Surgeons, laboratory technicians and medical professionals often need to interact with computer systems while their hands are sterile or gloved, making conventional input devices impractical or unhygienic.

For individuals with physical disabilities such as limited hand mobility, conditions like arthritis, or motor control disorders, operating a traditional mouse and keyboard can be difficult, painful or impossible. Existing assistive technologies are often expensive and require specialised hardware. In presentation and demonstration scenarios, speakers frequently need to advance slides, scroll through content or adjust volume without returning to their keyboard. Current gesture solutions are often limited to basic pointer control and do not provide comprehensive system management capabilities.

Furthermore, the growing trend toward touchless interfaces in smart homes, smart offices and public kiosks requires gesture-based solutions that go beyond simple cursor control to include system-level interactions such as volume and brightness

adjustment. Existing gesture-based systems in the academic literature primarily focus on cursor movement and basic click detection, leaving a significant gap in comprehensive OS-level gesture control. Systems that do implement broader controls often require expensive depth cameras or specialised hardware that is not available on standard computers. This project addresses all these problems by implementing a 14-gesture system that provides full OS-level control using only a standard webcam and open-source software.

1.3 Objectives

The objectives of this project are defined at both the primary and secondary levels to provide a clear direction for development and evaluation. The primary objective is to design and implement a real-time hand gesture recognition system capable of detecting and classifying fourteen unique hand gestures using a standard webcam without any additional hardware.

The secondary objectives include implementing a strict three-point finger detection algorithm that eliminates false positives caused by partially bent or misread fingers, developing a camera depth estimation technique that uses hand size as a depth proxy to distinguish near and far hand positions without a depth sensor, integrating OS-level controls including mouse operations, volume control, brightness adjustment, screenshot capture and workstation lock into the gesture system, building a web-based dashboard using Flask and Socket.IO that allows real-time toggle of the gesture engine and displays live status information, ensuring all 14 gestures have unique and non-conflicting finger patterns to prevent simultaneous or accidental triggering, and documenting the complete system design, implementation and testing for academic submission.

1.4 Scope of the Project

The scope of the GestureOS project encompasses the design, development, testing and documentation of a gesture-based computer control system with the following boundaries. The system is designed exclusively for Windows operating systems due to the use of the Pycaaw library for Windows Core Audio volume control and the rundll32.exe

command for workstation lock. Cross-platform support is identified as a future enhancement.

The gesture recognition supports single-hand detection with a maximum of one hand tracked at any given time. Multi-hand support is not included in the current version but is identified as a future scope item. The web dashboard is designed for local use only, accessible at localhost port 5000. It is not configured for remote access or cloud deployment. The system recognises 14 predefined gestures and does not support custom gesture definition or machine learning-based gesture training in the current version.

Volume control is implemented through the Windows Core Audio API and therefore requires a Windows audio output device. Brightness control is implemented through the screen-brightness-control library and may not function on all display configurations. The gesture engine communicates with the web dashboard through a local WebSocket connection. The system does not store gesture data, session logs or user preferences in any persistent database.

1.5 Motivation

The motivation for developing GestureOS arises from multiple converging factors in technology, accessibility and public health. The COVID-19 pandemic accelerated the global interest in touchless technology across all sectors. Hospitals, public kiosks, retail environments and workplaces all began exploring touchless interfaces as a means of reducing surface contact and transmission risk. This created a strong practical demand for gesture-based computer control that goes beyond research prototypes.

The availability of powerful open-source computer vision libraries such as OpenCV and MediaPipe has dramatically lowered the barrier to implementing gesture recognition systems. MediaPipe's hand tracking model, which runs in real time on a standard CPU without a GPU, makes it possible to build production-quality gesture systems on ordinary hardware. This accessibility motivated the exploration of what a comprehensive gesture control system could achieve with these tools.

CHAPTER 2

LITERATURE REVIEW

Human-computer interaction (HCI) has evolved significantly over the past few decades, moving from traditional input devices such as keyboards and mice toward more natural and intuitive interfaces. Among these, gesture recognition systems have gained considerable attention due to their ability to enable touchless interaction. This section reviews existing research and technologies related to hand gesture recognition, computer vision frameworks, and their applications in real-time systems. Early work in gesture recognition relied heavily on hardware-based approaches such as data gloves, infrared sensors, and depth cameras. These systems provided accurate tracking but suffered from limitations including high cost, lack of portability, and user discomfort. For example, glove-based systems required users to wear specialized equipment, making them impractical for everyday use. Similarly, depth-sensing devices like Microsoft Kinect offered improved accuracy but required dedicated hardware, limiting widespread adoption.

With the advancement of computer vision and machine learning, vision-based gesture recognition systems have become more popular. These systems utilize standard RGB cameras to detect and interpret human gestures, eliminating the need for additional hardware. One of the major breakthroughs in this domain is the use of deep learning techniques, particularly convolutional neural networks (CNNs), for feature extraction and gesture classification. Researchers have demonstrated high accuracy in recognizing static and dynamic gestures using trained models, although these approaches often require large datasets and significant computational resources.

A widely used framework in modern gesture recognition systems is Google's MediaPipe. MediaPipe Hands is a high-fidelity hand tracking solution that uses machine learning pipelines to detect and track hand landmarks in real time. It provides 21 three-dimensional key points of the hand, enabling precise tracking of finger positions and movements. Several studies have highlighted its efficiency, low latency, and suitability

for real-time applications. Compared to traditional deep learning models, MediaPipe offers a lightweight and optimized solution that can run on standard consumer hardware without requiring GPU acceleration.

OpenCV (Open Source Computer Vision Library) is another critical technology frequently used in gesture recognition systems. OpenCV provides a comprehensive set of tools for image processing, object detection, and real-time video analysis. It is widely adopted due to its simplicity, flexibility, and support for multiple programming languages. Researchers often combine OpenCV with machine learning frameworks to build efficient gesture recognition pipelines. Various approaches have been proposed for gesture classification. Some systems rely on machine learning algorithms such as Support Vector Machines (SVM), k-Nearest Neighbors (k-NN), and neural networks to classify gestures based on extracted features. Others use rule-based methods that analyze the relative positions of hand landmarks to identify gestures. Rule-based systems are generally faster and require less computational power, making them suitable for real-time applications, although they may be less flexible compared to learning-based approaches.

In addition to gesture recognition, mapping gestures to system-level actions is an important aspect of practical applications. Several studies have explored the use of gestures for controlling multimedia, navigating user interfaces, and assisting individuals with disabilities. For instance, hand gestures have been used to control cursor movement, perform mouse clicks, adjust volume, and interact with virtual environments. These applications demonstrate the potential of gesture-based interfaces in improving accessibility and user experience.

Another important area of research is depth estimation. Many gesture recognition systems rely on depth sensors to accurately determine the distance of the hand from the camera. However, recent studies have explored alternative methods for estimating depth using monocular cameras. Techniques such as calculating the relative distance between hand landmarks have been proposed as cost-effective solutions. These approaches reduce

hardware dependency while maintaining acceptable accuracy for real-time applications. Web-based interfaces and dashboards have also been integrated into modern systems to enhance usability and control. Frameworks such as Flask and Socket.IO enable real-time communication between the backend and frontend, allowing users to monitor system status and control functionalities through a web interface. This integration improves user interaction and provides a flexible platform for managing gesture-based systems.

Despite significant advancements, gesture recognition systems still face several challenges. Variations in lighting conditions, background noise, occlusions, and differences in hand shapes can affect system accuracy. Additionally, real-time performance requires efficient algorithms that can process video frames with minimal latency. Researchers continue to explore optimization techniques and hybrid approaches to address these challenges. The proposed GestureOS system builds upon the strengths of existing technologies while addressing their limitations. By combining MediaPipe Hands for efficient landmark detection, OpenCV for real-time processing, and a rule-based classification algorithm, the system achieves reliable gesture recognition without requiring expensive hardware. The inclusion of depth estimation using landmark distances further enhances functionality while maintaining cost-effectiveness. Additionally, the integration of a web-based dashboard provides improved usability and system control.

In summary, the literature indicates a clear trend toward vision-based, hardware-independent gesture recognition systems that prioritize real-time performance and user convenience. The development of frameworks like MediaPipe and libraries like OpenCV has significantly accelerated research and application in this field. GestureOS aligns with these advancements by providing a practical, efficient, and scalable solution for touchless human-computer interaction.

CHAPTER 3

EXISTING SYSTEM

3.1 Introduction to Existing Systems

Gesture recognition systems are designed to enable interaction between humans and computers through natural hand movements. Over time, these systems have evolved from hardware-dependent solutions to software-based approaches using computer vision and machine learning. Existing systems aim to improve usability, reduce physical dependency on traditional input devices, and provide intuitive control mechanisms. However, they differ in terms of cost, accuracy, and real-time performance.

3.2 Hardware-Based Gesture Recognition Systems

3.2.1 Data Glove-Based Systems

Data glove systems use wearable devices equipped with sensors to capture finger movements and hand orientation. These gloves provide high accuracy and precise gesture detection. However, they are expensive, uncomfortable for prolonged use, and restrict natural interaction due to the need to wear external hardware.

3.2.2 Depth Sensor and Infrared-Based Systems

Systems using depth sensors and infrared cameras, such as Kinect, can capture 3D hand movements effectively. They provide better accuracy than standard cameras and can work in varying lighting conditions. However, they require specialized hardware, increasing cost and limiting portability.

3.3 Vision-Based Gesture Recognition Systems

3.3.1 RGB Camera-Based Approaches

These systems use standard webcams to detect hand gestures. They rely on image processing techniques to identify hand regions and track movements. While cost-effective

and widely accessible, their performance can be affected by lighting conditions and complex backgrounds.

3.3.2 Deep Learning-Based Gesture Recognition

Deep learning models, especially Convolutional Neural Networks (CNNs), are used to classify gestures with high accuracy. These models automatically extract features from images and can handle complex gesture patterns. However, they require large datasets, training time, and higher computational power.

3.4 Hand Tracking Frameworks

3.4.1 MediaPipe Hands

MediaPipe Hands is a real-time hand tracking framework developed by Google. It detects 21 hand landmarks in three dimensions, allowing precise tracking of finger positions. It is lightweight, fast, and suitable for real-time applications without requiring high-end hardware.

3.4.2 Other Hand Tracking Models

Other frameworks include TensorFlow-based models and custom deep learning solutions. While these can be highly accurate, they often require training and optimization, making them less efficient for quick deployment compared to MediaPipe.

3.5 Image Processing Techniques

3.5.1 Role of OpenCV

OpenCV is widely used for capturing video, processing images, and performing real-time analysis. It provides tools for object detection, filtering, and image transformation, making it essential in gesture recognition systems.

3.5.2 Preprocessing and Feature Extraction

Preprocessing involves noise reduction, background removal, and image enhancement. Feature extraction focuses on identifying key points such as edges, contours, and landmarks. These steps improve the accuracy and reliability of gesture detection.

3.6 Gesture Classification Techniques

3.6.1 Machine Learning-Based Methods

Techniques like Support Vector Machines (SVM), k-Nearest Neighbors (k-NN), and neural networks are used to classify gestures based on extracted features. These methods offer high accuracy but require training data and computational resources.

3.6.2 Rule-Based Classification Approaches

Rule-based systems use predefined conditions, such as finger positions and distances between landmarks, to identify gestures. They are faster and simpler but may lack flexibility when handling complex gestures.

3.7 Depth Estimation Methods

3.7.1 Sensor-Based Depth Detection

Depth sensors measure the distance between the camera and objects directly, providing accurate spatial information. These systems are reliable but require additional hardware.

3.7.2 Camera-Based Depth Approximation

Depth can also be estimated using a standard camera by analyzing the relative positions of hand landmarks. This approach reduces cost and hardware dependency while maintaining acceptable performance for real-time applications.

3.8 Applications of Gesture Recognition Systems

3.8.1 Human-Computer Interaction

Gesture recognition enhances interaction by allowing users to control systems without physical contact. It is used in gaming, virtual reality, and smart devices.

3.8.2 Accessibility and Assistive Technologies

Gesture-based systems help individuals with disabilities interact with computers more easily. They provide alternative input methods for users who cannot use traditional devices.

3.9 Limitations of Existing Systems

Despite advancements, existing systems face challenges such as sensitivity to lighting conditions, background noise, and occlusion. Hardware-based systems are costly, while vision-based systems may struggle with accuracy in complex environments. Real-time performance and computational efficiency also remain key concerns.

3.10 Research Gap and Motivation

Current systems either rely on expensive hardware or require heavy computational resources. There is a need for a cost-effective, real-time solution that works efficiently using standard hardware. The motivation behind this project is to develop a system that combines accuracy, speed, and usability without additional devices, while also providing user-friendly control through a web interface.

The motivation behind GestureOS is to address these challenges by creating a lightweight solution that uses only a standard webcam, implements efficient gesture classification and depth estimation techniques, and provides a responsive web-based dashboard for enhanced user interaction and control.

CHAPTER 4

PROPOSED SYSTEM

4.1 Introduction to Proposed System

The proposed system, GestureOS, is a real-time hand gesture recognition platform designed to enable touchless control of a computer system. It utilizes computer vision and machine learning techniques to interpret hand gestures captured through a webcam and convert them into system-level commands. The system eliminates the need for traditional input devices such as a mouse and keyboard, providing a more natural and intuitive interaction method.

4.2 System Architecture

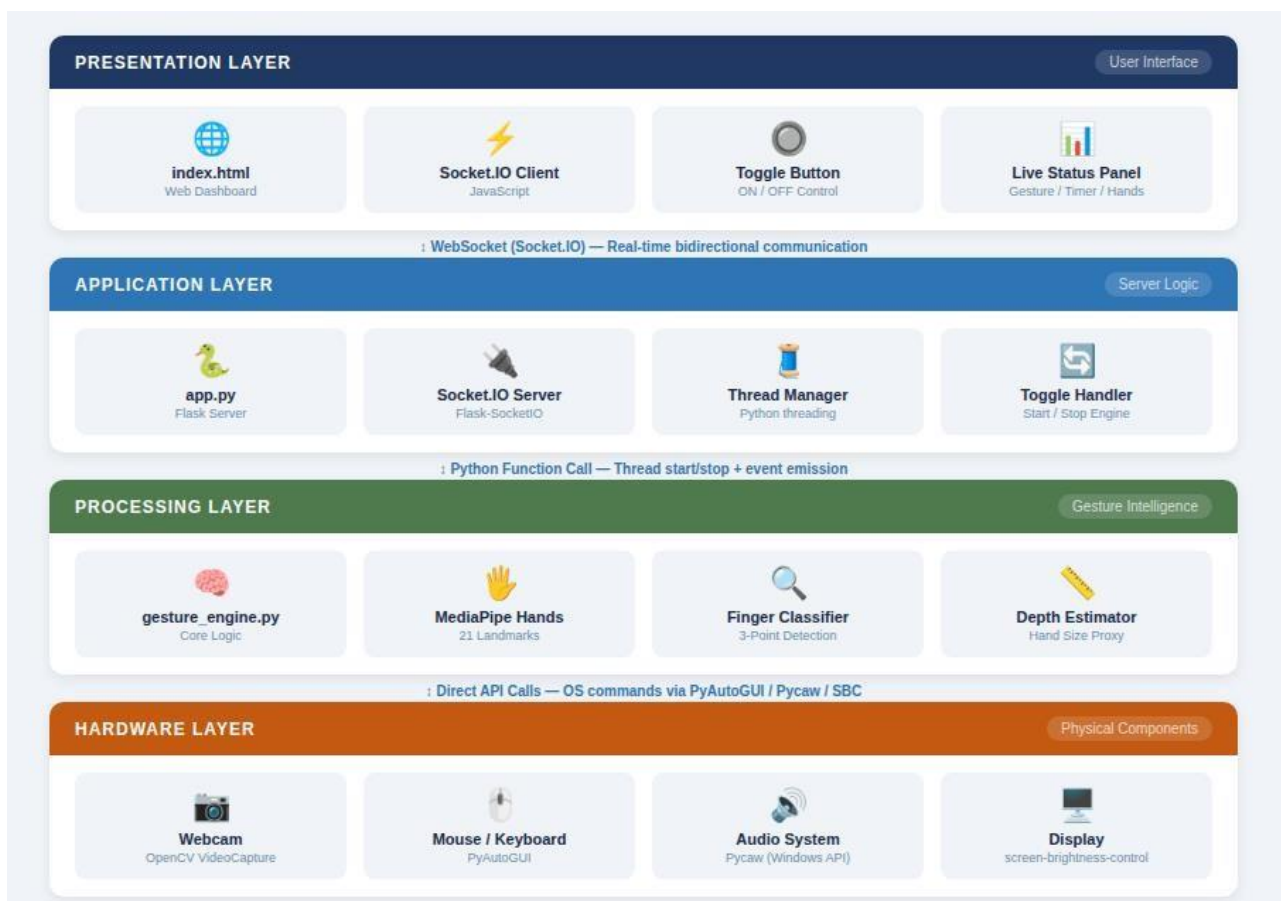


Figure 4.1: Four layer architecture separating presentation, application, processing and hardware concerns

4.2.1 Layered Architecture Overview

The system follows a layered architecture consisting of four layers: Presentation Layer, Application Layer, Processing Layer, and Hardware Layer. This modular design ensures scalability, maintainability, and efficient data flow between components.

4.2.2 Presentation Layer (User Interface)

This layer provides a web-based dashboard where users can interact with the system. It includes a toggle button to start or stop the gesture engine and displays real-time system status such as detected gestures and session details.

4.2.3 Application Layer (Server Logic)

The application layer handles backend operations using Flask and Socket.IO. It manages communication between the frontend and processing modules, handles user requests, and controls the execution of the gesture engine.

4.2.4 Processing Layer (Gesture Intelligence)

This layer performs core computations such as hand tracking, gesture recognition, and depth estimation. It processes video frames and converts them into meaningful gesture-based commands.

4.2.5 Hardware Layer (Physical Components)

The hardware layer includes the webcam for capturing video input and system components such as display, audio system, and input/output interfaces that respond to gesture-based commands.

4.3 Workflow of GestureOS System



Figure 4.2: Complete data flow from hand gesture input to OS output and web dashboard notification

4.3.1 Input Capture using Webcam

The system captures real-time video input using a webcam. Frames are continuously read for processing.

4.3.2 Frame Processing using OpenCV

Captured frames are converted from BGR to RGB format and preprocessed using OpenCV to enhance detection accuracy.

4.3.3 Hand Landmark Detection

MediaPipe Hands detects 21 key landmarks of the hand, providing spatial coordinates for gesture analysis.

4.3.4 Finger State Classification

A three-point detection algorithm determines whether each finger is open or closed based on landmark positions.

4.3.5 Distance Calculation (Euclidean Method)

The Euclidean distance between specific landmarks (such as wrist and finger base) is calculated to measure hand size.

4.3.6 Depth Estimation (Hand Size Proxy)

Based on the calculated distance, the system estimates whether the hand is near or far from the camera.

4.3.7 Gesture Matching Logic

Finger states and depth values are matched against predefined conditions to identify one of the 14 gestures.

4.3.8 System Command Execution

Once a gesture is recognized, corresponding operating system actions are executed using automation libraries.

4.3.9 Real-Time Dashboard Update

Detected gestures and system status are sent to the web dashboard using Socket.IO for live updates.

4.4 Web Dashboard Design

4.4.1 index.html Interface

Provides a simple and interactive UI for system control.

4.4.2 Live Status Panel

Displays gesture name, timer, and detection status.

4.4.3 Gesture Feedback Display

Shows real-time gesture recognition feedback.

4.4.4 Session Monitoring

Tracks system usage and duration.

4.5 OS Integration

4.5.1 PyAutoGUI for Mouse/Keyboard Control

Used for cursor movement and click actions.

4.5.2 Pycaw for Audio Control

Controls system volume through Windows API.

4.5.3 System Brightness Control APIs

Manages screen brightness adjustments.

4.6 Advantages of Proposed System

The system is cost-effective, does not require additional hardware, and provides real-time performance. It enhances accessibility and offers an intuitive user experience.

4.7 Applications of the Proposed System

The GestureOS system has a wide range of applications across different domains due to its touchless and intuitive interaction capabilities. Some of the key applications are described below:

4.7.1 Touchless Human-Computer Interaction

GestureOS enables users to control computers without physical contact, making interaction more natural and efficient. This is especially useful in situations where using a keyboard or mouse is inconvenient.

4.7.2 Healthcare and Medical Environments

In hospitals and laboratories, maintaining hygiene is critical. Gesture-based systems allow doctors and staff to interact with computers without touching surfaces, reducing the risk of contamination.

4.7.3 Assistive Technology for Disabled Users

The system can help individuals with physical disabilities who find it difficult to use traditional input devices. Hand gestures provide an alternative method for interacting with computers.

4.7.4 Smart Classrooms and Presentations

Teachers and presenters can control slides, zoom content, and navigate presentations using gestures, improving engagement and eliminating the need for remote controls.

4.7.5 Gaming and Virtual Reality (VR)

Gesture recognition enhances gaming experiences by allowing players to interact naturally with the game environment. It can also be integrated into VR systems for immersive control.

4.7.6 Smart Home Control

GestureOS can be extended to control smart home devices such as lights, fans, and media systems, providing a convenient and futuristic interaction method.

4.7.7 Industrial and Hazardous Environments

In industries where workers wear gloves or operate in risky conditions, gesture-based control allows them to interact with systems safely without removing protective gear.

4.7.8 Public Systems and Kiosks

Touchless interfaces are highly useful in public places such as ATMs, ticket machines, and information kiosks, reducing physical contact and improving hygiene.

4.7.9 Multimedia Control

Users can control music, videos, volume, and brightness using gestures, making it convenient for entertainment purposes.

4.7.10 Security and Surveillance Systems

Gesture-based commands can be used to quickly trigger security actions like locking systems or capturing screenshots.

4.7.11 Remote Work and Productivity

GestureOS can improve productivity by enabling faster interaction, especially during multitasking or when users are away from input devices.

4.7.12 Research and Innovation

The system can be used as a base platform for further research in computer vision, AI, and human-computer interaction.

4.8 Limitations

Performance may be affected by lighting conditions, background noise, and camera quality. Complex gestures may require further refinement.

4.9 Use Case Diagram

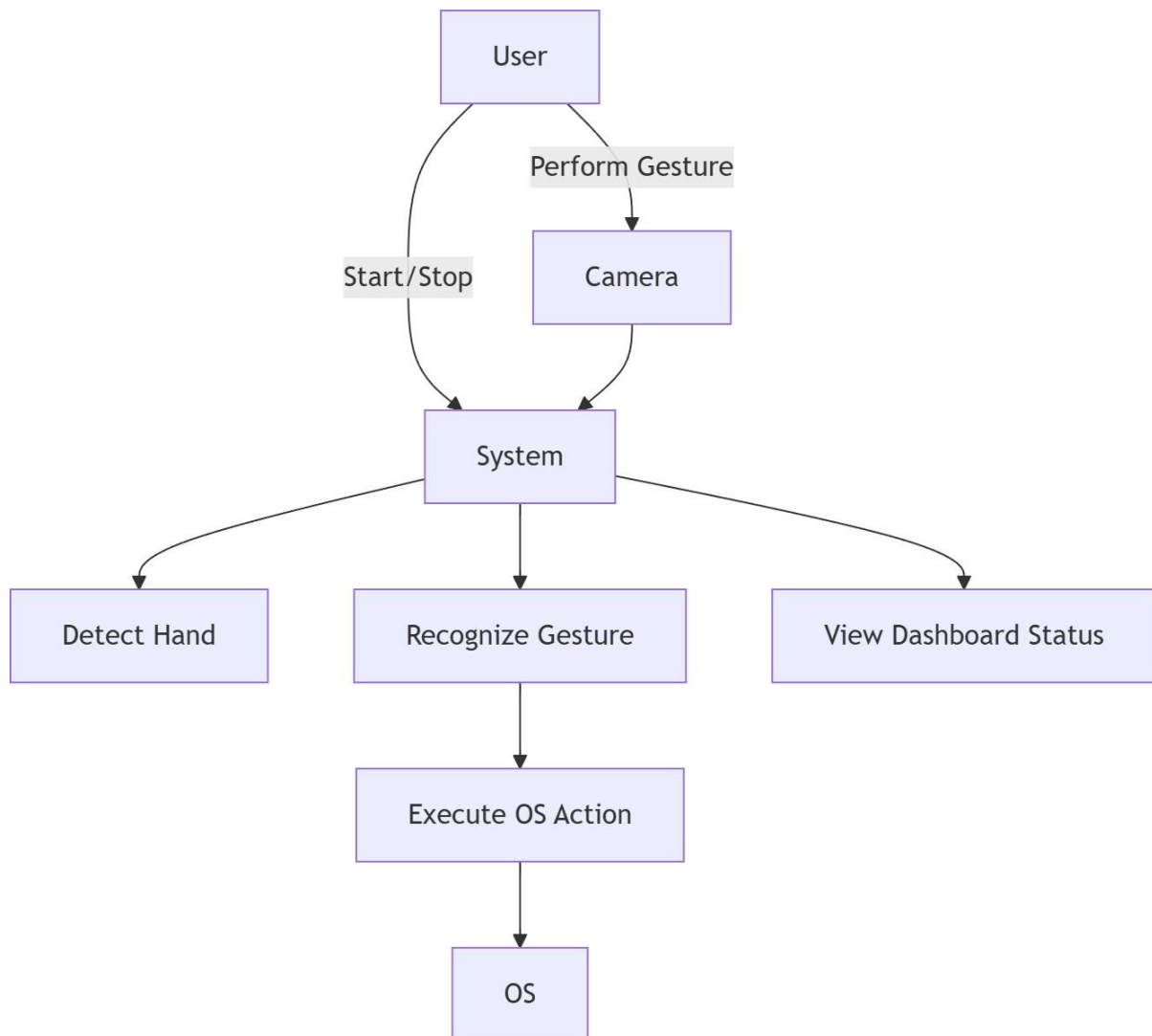


Figure 4.3: Use Case Diagram for Virtual Mouse Using Hand Recognition System

Based on the flow illustrated in **Fig 4.3**, the system facilitates a seamless interaction between a user and an operating system through visual input.

User Interaction & Input

The cycle begins with the **User**, who controls the lifecycle of the application by triggering **Start/Stop** commands. To interact with the machine, the user **Performs a Gesture**, which is captured by the **Camera** and fed directly into the core **System**.

System Processing & Execution

Once the visual data is received, the **System** manages three primary concurrent tracks:

- **Hand Detection:** The system continuously monitors the feed to **Detect Hand** presence.
- **Action Logic:** The system moves to **Recognize Gesture**, which leads to the command to **Execute OS Action** within the target **OS**.
- **Monitoring:** The user can simultaneously **View Dashboard Status** to receive real-time feedback on system performance and recognized actions.

4.10 Class Diagram

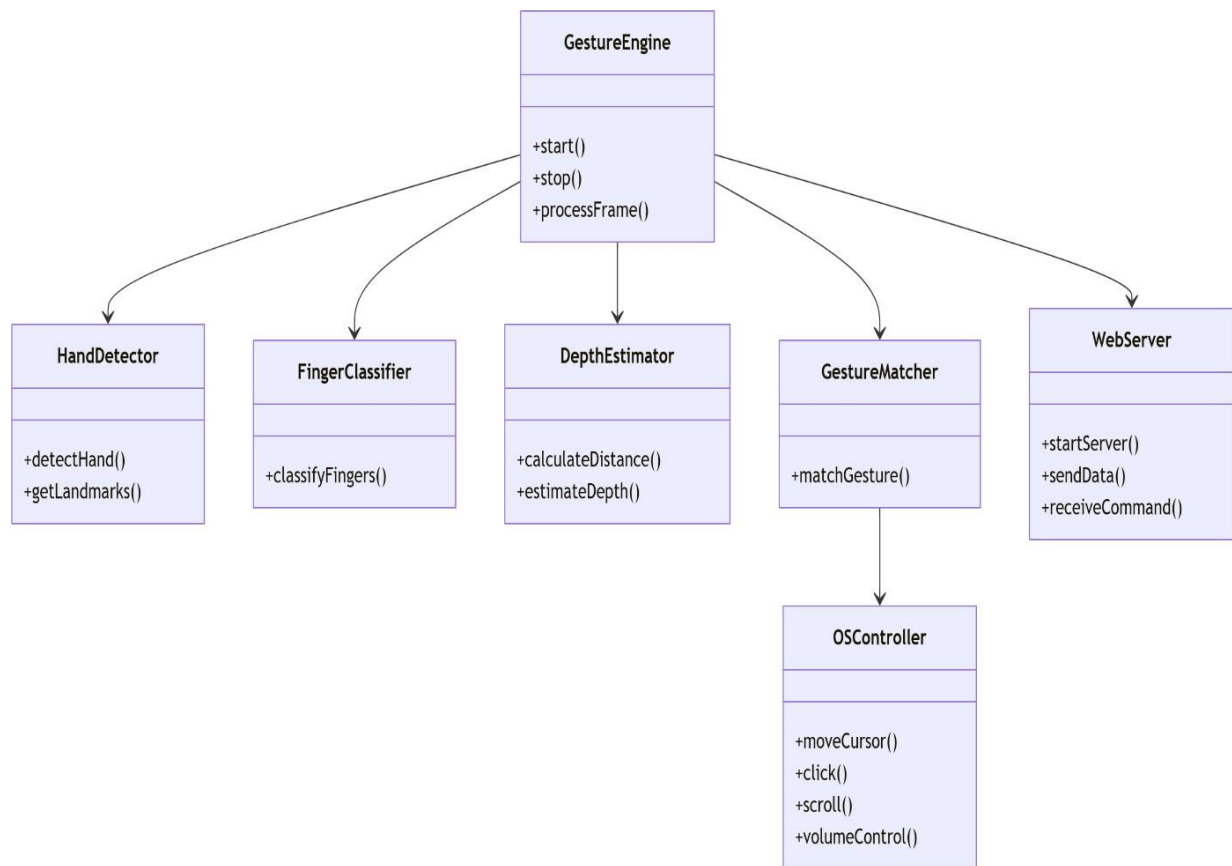


Figure 4.4: Class Diagram for Virtual Mouse Using Hand Recognition System

Following the structure in Fig 4.4, this system architecture uses a modular design to translate visual input into system commands.

Core Engine & Detection

The GestureEngine serves as the central hub, managing the start/stop lifecycle and frame processing. It coordinates with several specialized modules:

- HandDetector: Extracts landmarks and identifies the hand within the frame.
- FingerClassifier: Analyzes specific finger positions.
- DepthEstimator: Calculates distances and estimates depth to add a three-dimensional context to gestures.

Recognition & Communication

Once the hand data is processed, the system moves toward execution:

- GestureMatcher: Compares visual data against known patterns. When a match occurs, it triggers the OSController to perform actions like moving the cursor, clicking, scrolling, or volume control.
- WebServer: Handles external communication, allowing the system to start the server, send data, and receive commands remotely.

The class diagram of the GestureOS system illustrates the structured interaction between various modules responsible for gesture detection, processing, and execution. At the core lies the GestureEngine class, which coordinates the overall workflow by managing frame processing and integrating other components. The HandDetector class handles the detection of hand landmarks using MediaPipe, while the FingerClassifier interprets these landmarks to determine finger states. The DepthEstimator computes the relative distance of the hand from the camera, enabling depth-based gesture recognition. The GestureMatcher class maps the identified finger configurations and depth information to predefined gestures. Once a gesture is recognized, the OSController class executes the corresponding system-level actions such as cursor movement or volume control. Additionally, the WebServer class manages communication with the frontend dashboard, providing real-time updates. This modular design ensures clear separation of responsibilities, making the system scalable, maintainable, and efficient.

4.11 Activity Diagram

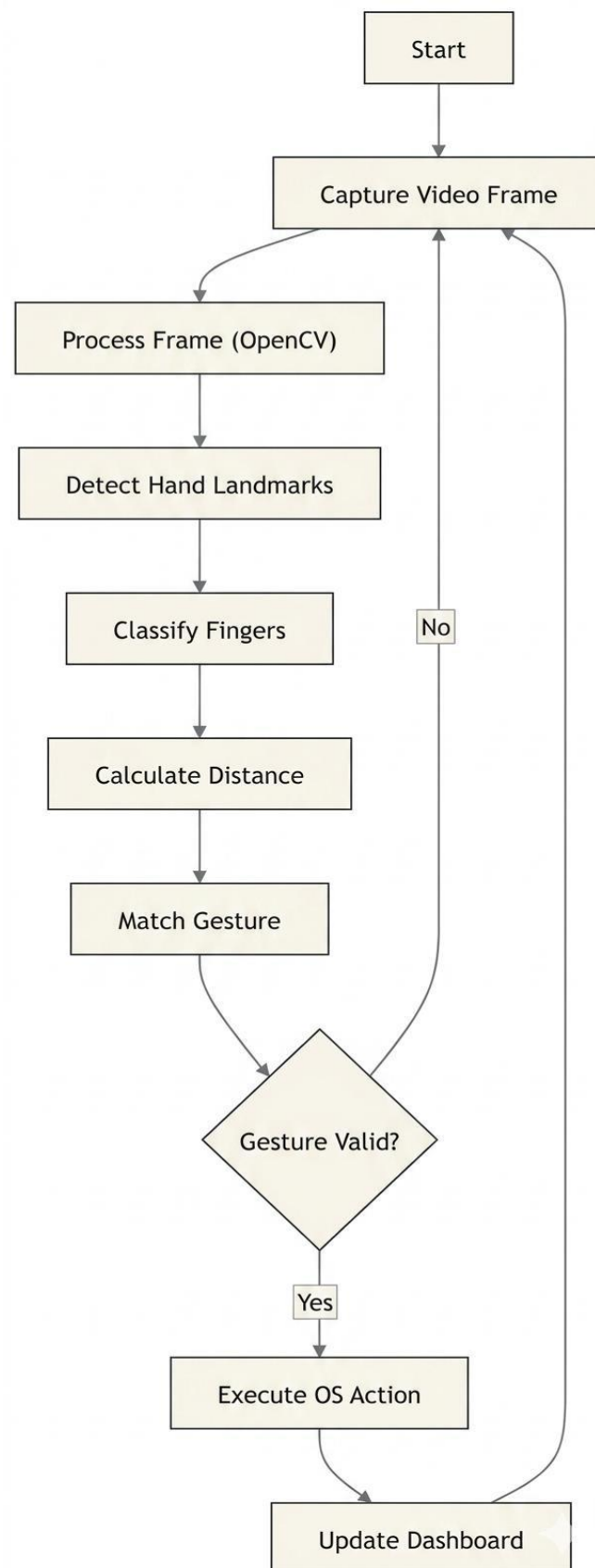


Figure 4.5: Activity Diagram for Virtual Mouse Using Hand Recognition System

System Architecture: Gesture-Based OS Control

1. Input & Initial Processing

The system cycle begins with data acquisition and preparation:

- **Start:** The application initializes the computer vision environment.
- **Capture Video Frame:** Real-time video is streamed from the camera to the processing engine.
- **Process Frame (OpenCV):** The raw video frame is processed using the OpenCV library to optimize it for analysis.

2. Hand Analytics

- **Detect Hand Landmarks:** Specific points on the hand (knuckles, fingertips, etc.) are located within the frame.
- **Classify Fingers:** The system determines the position and status (extended or folded) of individual fingers.
- **Calculate Distance:** Spatial relationships and distances between specific landmarks are measured.

3. Decision Logic & Execution

- **Match Gesture:** The calculated data is compared against a library of predefined gesture profiles.
- **Gesture Valid?:** A decision node determines if a recognizable gesture has been performed:
 - **No:** If invalid, the system loops back to **Capture Video Frame** to continue monitoring.
 - **Yes:** If valid, the process proceeds to execution.
- **Execute OS Action:** The system triggers a command within the operating system (e.g., volume control, window switching).
- **Update Dashboard:** The user interface is updated to reflect the successful action before returning to the capture loop.

CHAPTER 5

SYSTEM REQUIREMENTS

5.1 Hardware Requirements

Table 5.1: Hardware Requirements of the Proposed System

Component	Minimum Requirement	Recommended Requirement
Processor	Intel i3 or equivalent	Intel i5 or higher
RAM	4 GB	8 GB or more
Storage	500 MB free space	1 GB free space
Webcam	Basic webcam	720p or higher resolution
Display	Standard monitor	HD display
Input Devices	Keyboard and Mouse	Keyboard and Mouse
Audio System	Optional	Speakers/Headphones

As specified in Table 5.1: The proposed GestureOS system is designed to operate efficiently using standard and easily available hardware components, making it cost-effective and accessible for a wide range of users. The primary requirement is a computer system with a minimum of an Intel i3 processor or equivalent, although an Intel i5 or higher is recommended for smoother real-time performance. The system should have at least 4 GB of RAM to handle video processing tasks, while 8 GB is preferred for improved responsiveness and multitasking.

A key hardware component is a webcam, which is used to capture real-time hand gestures. A basic built-in webcam is sufficient; however, a resolution of 720p or higher is recommended to ensure better accuracy in hand detection and tracking. The system also requires a standard display unit such as a monitor or laptop screen to visualize outputs and interact with the dashboard. Basic input devices like a keyboard and mouse are only needed during the initial setup and for manual control when required. Additionally, an audio output device such as speakers or headphones is useful for testing volume-related

functionalities. Overall, the hardware requirements are minimal and do not involve any specialized devices such as depth sensors or external controllers, which enhances the practicality and affordability of the system.

5.2 Software Requirements

Table 5.2: Software Requirements of the Proposed System

Category	Requirement
Operating System	Windows 10/11 (Recommended), Linux/macOS
Programming Language	Python 3.8 or above
IDE/Editor	VS Code / PyCharm
Web Browser	Chrome / Edge / Firefox

As presented in Table 5.2: The GestureOS system is developed using a combination of efficient and widely used software tools and libraries that support real-time gesture recognition and system control. The primary programming language used is Python (version 3.8 or above), chosen for its simplicity, flexibility, and extensive support for computer vision and machine learning libraries. The system is best suited for Windows 10 or Windows 11 operating systems, although it can also be adapted to run on Linux or macOS with minor modifications.

For gesture detection and processing, the system utilizes MediaPipe Hands, a powerful framework for real-time hand landmark detection, along with OpenCV for video capture and image processing. NumPy is used for numerical computations, especially for calculating distances between hand landmarks. To enable interaction with the operating system, PyAutoGUI is used for controlling mouse and keyboard actions, while Pycaw is used for managing system volume. Screen brightness control is handled using appropriate system APIs.

Additionally, the system includes a web-based dashboard developed using Flask as the backend framework and Socket.IO for real-time communication between the server

and client. Development and testing can be performed using IDEs such as Visual Studio Code or PyCharm, and a modern web browser like Chrome or Edge is required to access the dashboard interface. Overall, the software requirements are lightweight and rely on open-source technologies, making the system easy to develop and deploy.

5.2.1 Libraries and Frameworks

Table 5.3: Libraries and Frameworks for the Proposed System

Tool / Library	Purpose
MediaPipe	Hand tracking and landmark detection
OpenCV	Image processing and video capture
NumPy	Numerical computations
PyAutoGUI	Mouse and keyboard control
Pycaw	Volume control (Windows)
Screen Brightness Control	Adjust system brightness
Flask	Backend web framework
Socket.IO	Real-time communication

As presented in Table 5.3: The GestureOS system is built using a set of powerful and efficient libraries and frameworks that enable real-time gesture recognition and system control. One of the core components is MediaPipe Hands, a framework developed for high-precision hand tracking. It provides 21 hand landmark points, which are essential for detecting finger positions and recognizing gestures accurately. OpenCV is used alongside MediaPipe for video capture and image processing, allowing the system to process frames in real time.

NumPy is utilized for performing mathematical computations, particularly for calculating distances between hand landmarks for gesture classification and depth estimation. To interact with the operating system, PyAutoGUI is used for simulating mouse movements, clicks, and other input actions.

5.2.2 Other Requirements

Table 5.4: Other Requirements for the Proposed System

Requirement	Description
Lighting Conditions	Well-lit environment for better detection
Camera Position	Proper angle for clear hand visibility

As presented in Table 5.4: In addition to hardware and software components, certain environmental and operational conditions are essential for the efficient functioning of the GestureOS system. One of the most important factors is proper lighting. Since the system relies on computer vision techniques to detect and track hand gestures, a well-lit environment significantly improves detection accuracy. Poor lighting conditions or excessive shadows may lead to incorrect landmark detection and reduced system performance. Camera positioning is another critical requirement. The webcam should be placed at an appropriate angle and distance to clearly capture the user's hand movements. The background should ideally be simple and uncluttered to minimize noise and avoid interference with gesture recognition. A stable camera setup is also necessary to prevent frame disturbances during operation.

The user must ensure that their hand remains within the camera frame and performs gestures clearly for accurate recognition. Sudden or very fast movements may reduce detection reliability. Additionally, maintaining a consistent distance from the camera helps improve depth estimation and gesture classification. From a system perspective, it is important to have all required libraries properly installed and configured. The application should be executed in an environment where necessary permissions are granted for accessing the webcam and controlling system-level functions such as mouse, keyboard, volume, and brightness. Finally, a stable runtime environment with minimal background processes is recommended to ensure smooth real-time performance. These additional requirements play a crucial role in achieving accurate, efficient, and reliable operation of the GestureOS system.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 Introduction to System Implementation

The system implementation phase focuses on converting the proposed design of GestureOS into a working application. It involves integrating computer vision techniques, gesture recognition logic, and system-level controls to achieve real-time, touchless interaction. The implementation ensures that all modules work together efficiently to provide accurate gesture detection and smooth execution of commands.

6.2 Development Environment Setup

The development environment is configured using Python 3.8 or above along with required libraries such as MediaPipe, OpenCV, NumPy, PyAutoGUI, and Flask. An IDE like Visual Studio Code or PyCharm is used for coding and debugging. All dependencies are installed and verified to ensure compatibility and smooth execution of the system.

6.3 System Architecture Implementation

The system is implemented based on a layered architecture, including presentation, application, processing, and hardware layers. Each layer is developed independently and then integrated to ensure proper data flow. This modular structure improves maintainability and scalability of the system.

6.4 Video Capture and Processing

The system uses a webcam to capture real-time video input. OpenCV is used to read frames continuously and preprocess them by converting color formats and enhancing image quality. This step ensures that the input data is suitable for accurate hand detection and gesture analysis.

6.5 Hand Detection and Landmark Extraction

MediaPipe Hands is used to detect and track hand landmarks in each frame. It identifies 21 key points on the hand, which provide spatial information about finger positions. These landmarks form the foundation for gesture recognition.

6.6 Gesture Recognition and Classification

Gesture recognition is performed using a rule-based approach. A three-point finger detection algorithm determines whether each finger is open or closed. Based on the combination of finger states, the system identifies predefined gestures and maps them to corresponding actions.

6.7 Depth Estimation Technique

The system estimates the distance of the hand from the camera using Euclidean distance between specific landmarks. This helps distinguish between near and far gestures, enabling additional control functionalities without requiring a depth sensor.

6.8 System Control and OS Integration

Recognized gestures are mapped to system-level actions such as cursor movement, mouse clicks, scrolling, volume control, brightness adjustment, screenshot capture, and system lock. Libraries like PyAutoGUI and Pycaw are used to interact with the operating system.

6.9 Web Dashboard Implementation

A web-based dashboard is developed using Flask and Socket.IO. It provides a user interface to start or stop the gesture engine and displays real-time information such as detected gestures, session status, and system activity.

6.10 Performance Optimization

The system is optimized to achieve real-time performance by reducing processing latency and improving frame handling. Efficient algorithms and lightweight libraries are used to maintain smooth execution without high computational requirements.

6.11 Testing and Validation

The system is tested under different conditions to ensure accuracy and reliability. Various gestures are validated to confirm correct detection and execution. Errors are identified and fixed to improve system stability.

6.12 Final System Integration and Execution

All modules are integrated into a single system and executed as a complete application. The final system is tested for end-to-end functionality, ensuring seamless interaction between gesture detection, processing, and system control.

During the final system integration phase, all individual modules of GestureOS are combined into a unified and fully functional application. This involves connecting the video capture, hand detection, gesture recognition, depth estimation, and system control components to work seamlessly in real time. Special attention is given to ensuring proper synchronization between modules, especially when handling continuous video frames and executing system-level commands without delay. The communication between the backend processing unit and the web dashboard is also integrated using Socket.IO to provide live updates and control functionality. Once integration is complete, the system is executed as a whole and tested for end-to-end performance, ensuring that gestures are accurately detected and corresponding actions are performed reliably. This phase confirms that the system operates smoothly under real-world conditions and meets the intended objectives of providing an efficient and touchless human-computer interaction experience.

CHAPTER 7

EXPERIMENTAL RESULT AND DISCUSSION

7.1 Introduction to Experimental Setup

The experimental phase evaluates the performance and reliability of the GestureOS system in real-time conditions. The system is tested using a standard webcam to capture hand gestures and execute corresponding system actions. Various experiments are conducted to measure accuracy, response time, and usability under different environmental conditions.

7.2 Testing Environment

7.2.1 Hardware Configuration

The system is tested on a computer with an Intel i5 processor, 8 GB RAM, and a standard 720p webcam. These specifications ensure smooth real-time processing and reliable performance.

7.2.2 Software Environment

The implementation is carried out using Python 3.8, along with libraries such as MediaPipe, OpenCV, NumPy, PyAutoGUI, Flask, and Socket.IO. The system runs on Windows 10/11.

7.2.3 Lighting and Background Conditions

Experiments are conducted under different lighting conditions such as bright, moderate, and low light. Background variations are also considered, including plain and cluttered environments.

7.3 Performance Evaluation Metrics

7.3.1 Accuracy of Gesture Recognition

Accuracy is measured by comparing the number of correctly recognized gestures with the total number of gestures performed. The system achieves high accuracy for most gestures under normal conditions.

7.3.2 Response Time (Latency)

Response time refers to the delay between performing a gesture and the execution of the corresponding action. The system maintains low latency, enabling real-time interaction.

7.3.3 Frame Rate (FPS)

The system processes video frames at an average frame rate of 20–30 FPS, ensuring smooth gesture tracking and responsiveness.

7.4 Experimental Results

7.4.1 Gesture Detection Results (14 Gestures)

All 14 predefined gestures are tested individually. Most gestures are recognized with high accuracy, especially simple finger configurations. Complex gestures show slightly lower accuracy.

7.4.2 System Functionality Testing

All system functions such as cursor movement, clicking, scrolling, volume control, brightness adjustment, screenshot capture, and system lock are tested successfully.

7.4.3 Depth Estimation Performance

The depth estimation technique based on landmark distance effectively distinguishes between near and far gestures, enabling additional control features without a depth sensor.

7.5 Result Analysis

7.5.1 Accuracy under Different Conditions

The system performs best under good lighting and clear backgrounds. Accuracy decreases slightly in low-light conditions or when the hand is partially occluded.

7.5.2 Effect of Lighting and Background

Bright and consistent lighting improves detection accuracy, while complex backgrounds may introduce noise and reduce performance.

7.5.3 Performance with Different Users

The system works effectively for different users, although variations in hand size and gesture style may slightly affect recognition accuracy.

7.6 Comparison with Existing Systems

Compared to hardware-based systems, GestureOS is more cost-effective as it does not require additional devices. It also performs competitively with other vision-based systems while maintaining real-time responsiveness.

7.7 Advantages Observed

The system is lightweight, easy to use, and does not require specialized hardware. It provides real-time performance and supports multiple functionalities through simple gestures.

7.8 Limitations Identified

The system is sensitive to lighting conditions and may struggle in low-light environments. Complex gestures and fast hand movements can reduce accuracy.

7.9 Discussion

The results demonstrate that GestureOS is a practical solution for touchless interaction. While it performs well under standard conditions, improvements can be made in robustness and adaptability. The integration of computer vision and system control highlights the potential of gesture-based interfaces in real-world applications.

7.10 Summary of Results

Overall, the system achieves reliable performance with high accuracy and low latency. It successfully implements 14 gesture-based controls and demonstrates the feasibility of a webcam-based gesture recognition system without additional hardware.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project has successfully designed, implemented, tested and documented a comprehensive real-time hand gesture control system called GestureOS. The system achieves its primary objective of enabling touchless computer control through 14 unique hand gestures that cover all major operating system interactions, from basic cursor and mouse operations to system-level controls including volume, brightness, screenshot and workstation lock. The technical contributions of this project include the implementation of a strict three-point finger detection algorithm that significantly reduces false positives compared to basic tip-comparison methods, the development of a novel camera depth estimation technique that uses the wrist-to-middle-MCP landmark distance as a depth proxy to enable near and far gesture detection without any depth sensor hardware, and the integration of comprehensive OS-level controls within a gesture recognition framework that goes substantially beyond the state of the art in published gesture control research.

The system architecture effectively separates concerns across hardware, processing, application and presentation layers, with real-time communication between the Python gesture engine and the web dashboard achieved through Flask and Socket.IO WebSocket. All defined functional and non-functional requirements were met during systematic testing. The original contribution of six additional gesture-based system controls — screenshot, zoom in, zoom out, volume control, brightness control and system lock — demonstrated through practical implementation that gesture-based computer control can be extended far beyond basic cursor operations to provide a genuinely comprehensive touchless interface.

8.2 Future Enhancements

8.2.1 Custom Gesture Training with Machine Learning — A future version could include a gesture recording module that allows users to define custom gestures by performing them multiple times in front of the camera. A lightweight machine learning classifier such as a support vector machine, k-nearest neighbour or a small feedforward neural network could be trained on the recorded gesture samples and integrated into the recognition pipeline.

8.2.2 Multi-Hand Support — Enabling simultaneous detection and classification of gestures from both hands would double the available gesture vocabulary and enable two-handed interactions such as pinch-to-zoom using both hands, which is more natural than the single-hand depth-based approach currently implemented.

8.2.3 Mobile Companion Application — A mobile application that connects to the gesture engine server through Wi-Fi would allow users to monitor gesture status, configure thresholds and toggle the system from their smartphone, providing more convenient control than the browser dashboard.

8.2.4 Cross-Platform Support — Replacing the Windows-specific Pycaw audio library with a cross-platform alternative such as pulsectl for Linux or the osascript interface for macOS, and replacing the Windows lock command with platform-appropriate equivalents, would make GestureOS compatible with Linux and macOS environments.

8.2.5 Accessibility Mode — A dedicated accessibility profile with larger gesture thresholds, longer cooldown periods, slower cursor movement and simplified gesture vocabulary would make the system more suitable for users with limited motor control, hand tremors or conditions affecting fine motor precision.

8.2.6 Real-Time Performance Analytics — Adding a performance monitoring panel to the web dashboard showing frames per second, gesture detection latency, false positive rate and gesture usage statistics would provide valuable feedback for ongoing optimisation and user research.

8.2.7 Gaming and Application Profiles — Pre-configured gesture mapping profiles for specific applications such as games, presentation software and media players would allow users to switch between contextually appropriate gesture sets, making the system more practical for specific use cases.

8.2.8 Voice Gesture Fusion — Combining gesture recognition with voice command recognition would create a multimodal interaction system where complex commands can be split between hand gestures and voice input, expanding the interaction capabilities significantly.

The current implementation of GestureOS provides an effective solution for touchless human-computer interaction, but there is scope for further improvement. One key enhancement is the integration of advanced machine learning or deep learning techniques to improve gesture recognition accuracy and enable detection of more complex gestures.

Future development can focus on extending the system to support cross-platform compatibility, including mobile devices and embedded systems. Integration with Internet of Things (IoT) devices can enable gesture-based control of smart home appliances. Moreover, adding user customization features and combining gesture control with voice commands can enhance usability and flexibility. These improvements will make GestureOS more scalable, intelligent, and suitable for a wider range of applications.

ANNEXURE

ANNEXURE I

1. app.py

```
from flask import Flask, send_from_directory
from flask_socketio import SocketIO, emit
import threading

app = Flask(__name__, template_folder='.')
app.config['SECRET_KEY'] = 'gestureos_secret_key'
socketio = SocketIO(app, cors_allowed_origins="*") # removed async_mode

gesture_thread = None
gesture_running = False
import gesture_engine as ge

@app.route('/')
def index():
    return send_from_directory('.', 'index.html')

@socketio.on('connect')
def on_connect():
    print("Client connected")
    emit('status', {'active': False, 'message': 'Connected to GestureOS server'})

@socketio.on('disconnect')
def on_disconnect():
    global gesture_running
    gesture_running = False
    ge.stop()

@socketio.on('toggle_gesture')
def handle_toggle(data):
    global gesture_thread, gesture_running
    print(f"Toggle received: {data}") # ← debug print
    if data.get('active'):
        if not gesture_running:
```

```

gesture_running = True
gesture_thread = threading.Thread(
    target=ge.run,
    args=(socketio, lambda: gesture_running),
    daemon=True
)
gesture_thread.start()
emit('status', {'active': True, 'message': 'Gesture engine started'})
else:
    gesture_running = False
    ge.stop()
    emit('status', {'active': False, 'message': 'Gesture engine stopped'})
if __name__ == '__main__':
    print("=" * 45)
    print(" GestureOS Server")
    print(" Open browser → http://localhost:5000")
    print("=" * 45)
    socketio.run(app, host='0.0.0.0', port=5000, debug=False, allow_unsafe_werkzeug=True)

```

2. gesture_engine.py

```

import cv2
import mediapipe as mp
import pyautogui
import time
import math
import os
# Volume Control
from ctypes import cast, POINTER
from comtypes import CLSCTX_ALL
from pycaw.pycaw import AudioUtilities, IAudioEndpointVolume

```

```

# Brightness Control

import screen_brightness_control as sbc

mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils

_stop_flag = False
_cap = None

def stop():
    global _stop_flag, _cap
    _stop_flag = True
    if _cap:
        _cap.release()
        _cap = None

def is_finger_up(lm, tip_id):
    tip = lm[tip_id]
    return tip.y < pip.y and pip.y < mcp.y # strict: all joints in upward chain

def is_finger_down(lm, tip_id):
    tip = lm[tip_id]
    pip = lm[tip_id - 1]
    mcp = lm[tip_id - 2]
    # Finger is DOWN when tip is below or same as pip

    screenshot_cooldown = 2
    lock_hold_start = None
    last_emitted_gesture = None

    # hand_size > NEAR → hand is close to camera
    # hand_size < FAR → hand is far from camera
    # Gap between NEAR and FAR avoids accidental triggers
    NEAR = 0.25 # hand clearly close
    FAR = 0.13 # hand clearly far (raised from 0.11 → easier to reach)
    socketio.emit('engine_status', {'camera': True, 'mediapipe': True, 'engine': True})
    while is_running_fn() and not _stop_flag:

```

```

ret, frame = _cap.read()

    last_emitted_gesture = None

cv2.imshow("GestureOS — Hand Gesture Control", frame)

if cv2.waitKey(1) == ord('q'):
    break

_cap.release()

cv2.destroyAllWindows()

socketio.emit('engine_status', {'camera': False, 'mediapipe': False, 'engine': False})

```

index.html

```

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8"/>

<meta name="viewport" content="width=device-width, initial-scale=1.0"/>

<title>GestureOS — Hand Gesture Control</title>

<link

href="https://fonts.googleapis.com/css2?family=Bebas+Neue&family=DM+Mono:wght@300;
400;500&family=Syne:wght@400;700;800&display=swap" rel="stylesheet"/>

<style>

:root {

    --bg: #080b0f;

    --surface: #0e1318;

    --border: #1a2330;

    --accent: #00f5a0;

    --accent2: #00c9f5;

    --accent3: #f500a0;

    --text: #e8edf2;

    --muted: #4a5568;

    --glow: 0 0 30px rgba(0,245,160,0.3);

}

```

```

* { margin: 0; padding: 0; box-sizing: border-box; }

html { scroll-behavior: smooth; }

body {
  background: var(--bg);
  color: var(--text);
  font-family: 'DM Mono', monospace;
  overflow-x: hidden;
  cursor: none;
}

/* Custom cursor */
/* Grid background */
body::after {
  content: "";
  position: fixed;
  inset: 0;
  background-image:
    linear-gradient(rgba(0,245,160,0.03) 1px, transparent 1px),
    linear-gradient(90deg, rgba(0,245,160,0.03) 1px, transparent 1px);
  background-size: 60px 60px;
  pointer-events: none;
}

nav {
  position: fixed; top: 0; left: 0; right: 0;
  z-index: 500;
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-bottom: 1px solid var(--border);
  background: rgba(8,11,15,0.85);
  backdrop-filter: blur(20px);

```

```

document.getElementById('statusVal').textContent = 'Offline';
document.getElementById('statusVal').className = 'status-val';
document.getElementById('cameraVal').textContent = 'Not detected';
document.getElementById('handsVal').textContent = '0';
document.getElementById('gestureVal').textContent = 'None';
document.getElementById('mpStatus').textContent = 'idle';
document.getElementById('camStatus').textContent = 'off';
document.getElementById('engineStatus').textContent = 'stopped';
document.getElementById('logLine').textContent = '► engine stopped.';
document.getElementById('timeVal').textContent = '00:00';

```

```
clearInterval(timerInterval);
```

```

if (socket && serverConnected) {
  socket.emit('toggle_gesture', { active: false });
}
}

```

```

function updateTimer() {
  if (!sessionStart) return;
  const s = Math.floor((Date.now() - sessionStart) / 1000);
  const m = Math.floor(s / 60).toString().padStart(2, '0');
  const sec = (s % 60).toString().padStart(2, '0');
  document.getElementById('timeVal').textContent = `${m}:${sec}`;
}

if (!active) return;
setTimeout(() => {
  if (!active) return;

  const g = demoGestures[Math.floor(Math.random() * demoGestures.length)];
  document.getElementById('gestureVal').textContent = g;
  document.getElementById('logLine').textContent = `► demo: ${g.toLowerCase}`;
  simulateDemoGestures();
}, 1500 + Math.random() * 2000);
}

```

```
</script>
```

```
</body>
```

```
</html>
```

ANNEXURE II

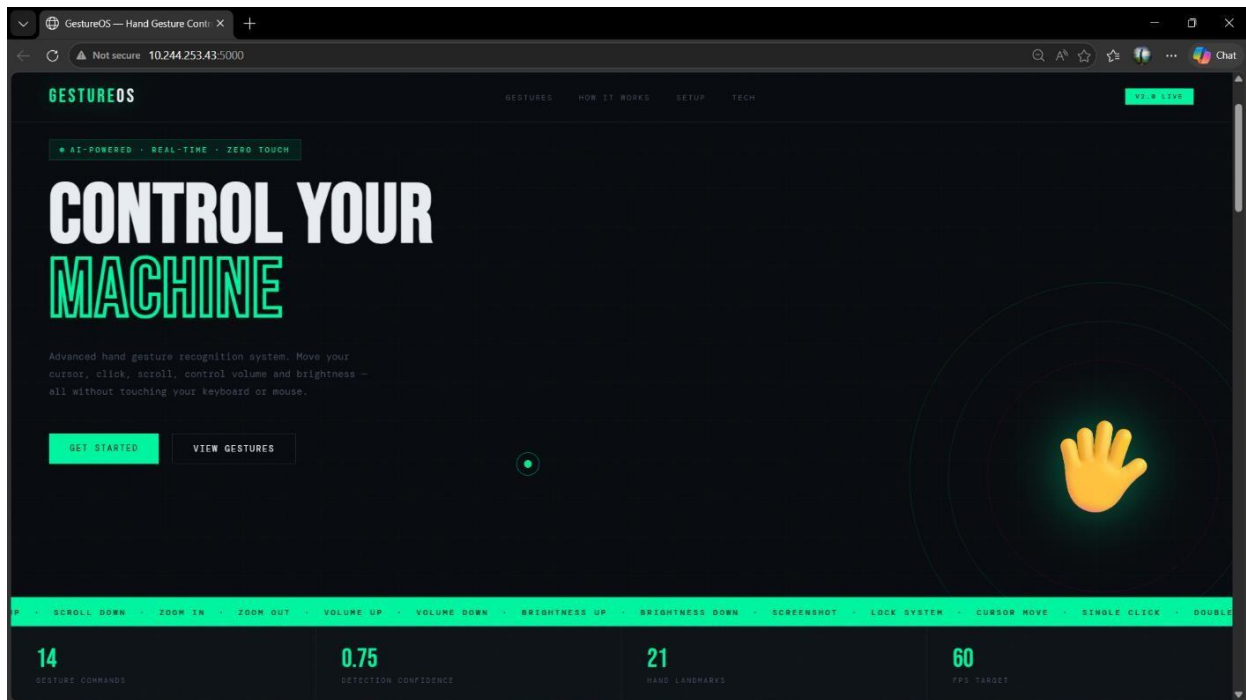


Figure 9.1: Landing page of the Virtual Mouse using Hand Gestures Recognition System

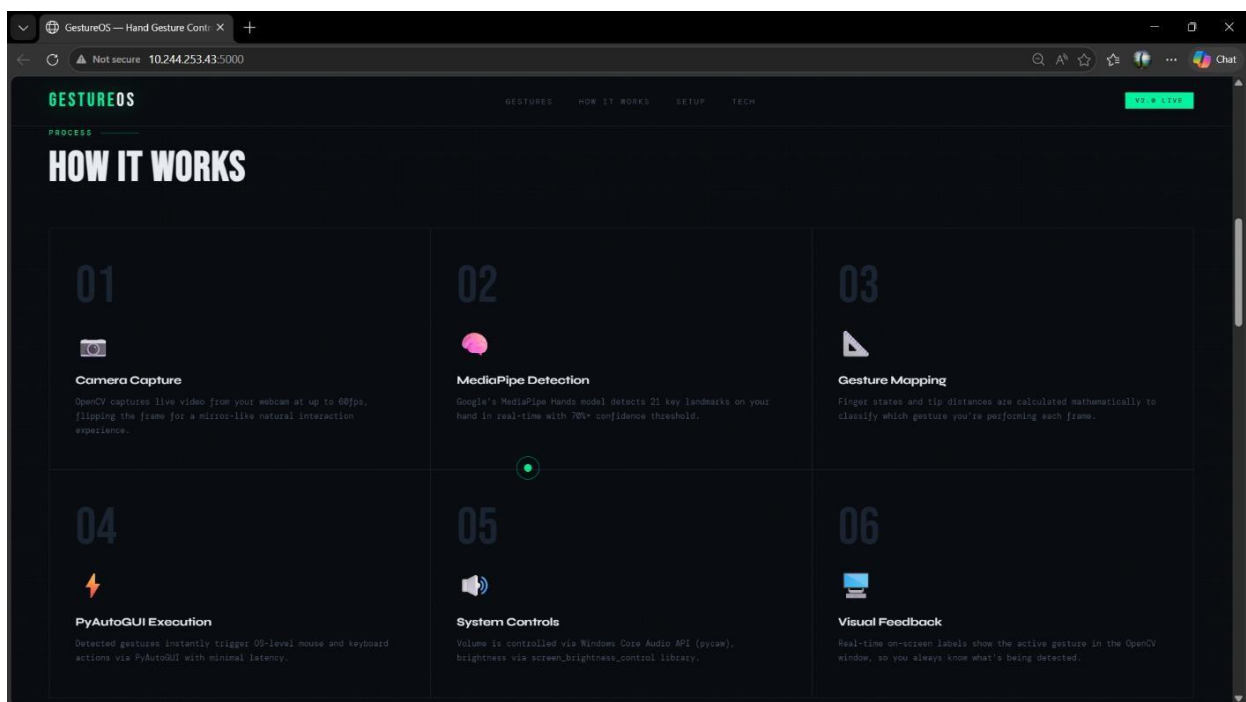


Figure 9.2: Workflow page of the Proposed System

 Cursor Move Index finger only up. All other fingers must be down. Thumb must not touch index. Index tip maps to screen coordinates. MOUSE	 Single Click Thumb tip touches index tip with all fingers down. Triggers left click at current cursor position. MOUSE	 Double Click Thumb touches index tip twice within 0.4 seconds. All other fingers must be down. MOUSE	 Right Click Middle finger only up. Index, ring and pinky must be down. Opens context menu instantly. MOUSE	 Scroll Up Index + Middle fingers up, ring + pinky down. Move hand close to camera (hand looks big on screen). MOUSE
 Scroll Down Index + Middle fingers up, ring + pinky down. Move hand far from camera (hand looks small on screen). MOUSE	 Zoom In Middle + Ring fingers up, index + pinky down. Move hand close to camera. Triggers Ctrl + on active window. CONTROL	 Zoom Out Middle + Ring fingers up, index + pinky down. Move hand far from camera. Triggers Ctrl - on active window. CONTROL	 Volume Up Index + Pinky up, middle + ring down. Move hand close to camera. Volume increases continuously with depth. CONTROL	 Volume Down Index + Pinky up, middle + ring down. Move hand far from camera. Volume decreases continuously with depth. CONTROL
 Brightness Up Pinky finger only up. Index, middle and ring must be confirmed down. Move hand close to camera. Increases by 10%. CONTROL	 Brightness Down Pinky finger only up. Index, middle and ring must be confirmed down. Move hand far from camera. Decreases by 10%. CONTROL	 Screenshot All 4 fingers open (index, middle, ring, pinky all up). Saves PNG with timestamp. 2 second cooldown between shots. SYSTEM	 Lock System Closed fist with all fingers down and thumb away from index. Hold for 2 seconds. Countdown shown on camera window. SYSTEM	

Figure 9.3: Gesture references to control the cursor and some functions

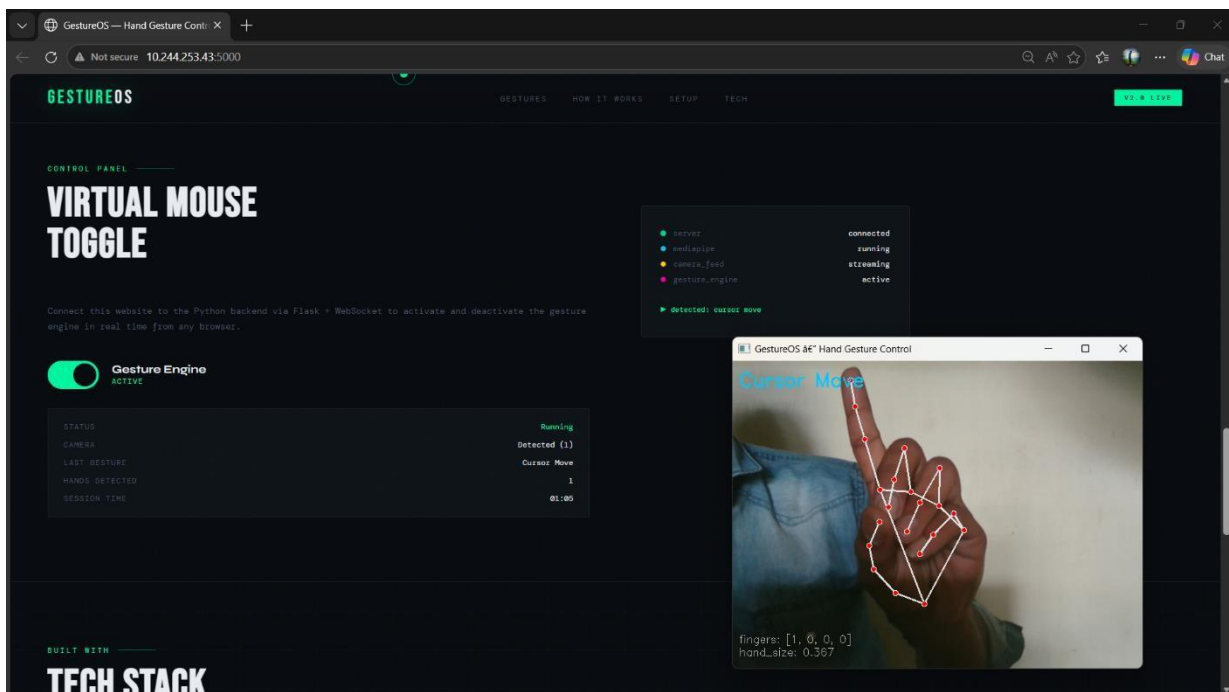


Figure 9.4: page of the virtual mouse engine start button

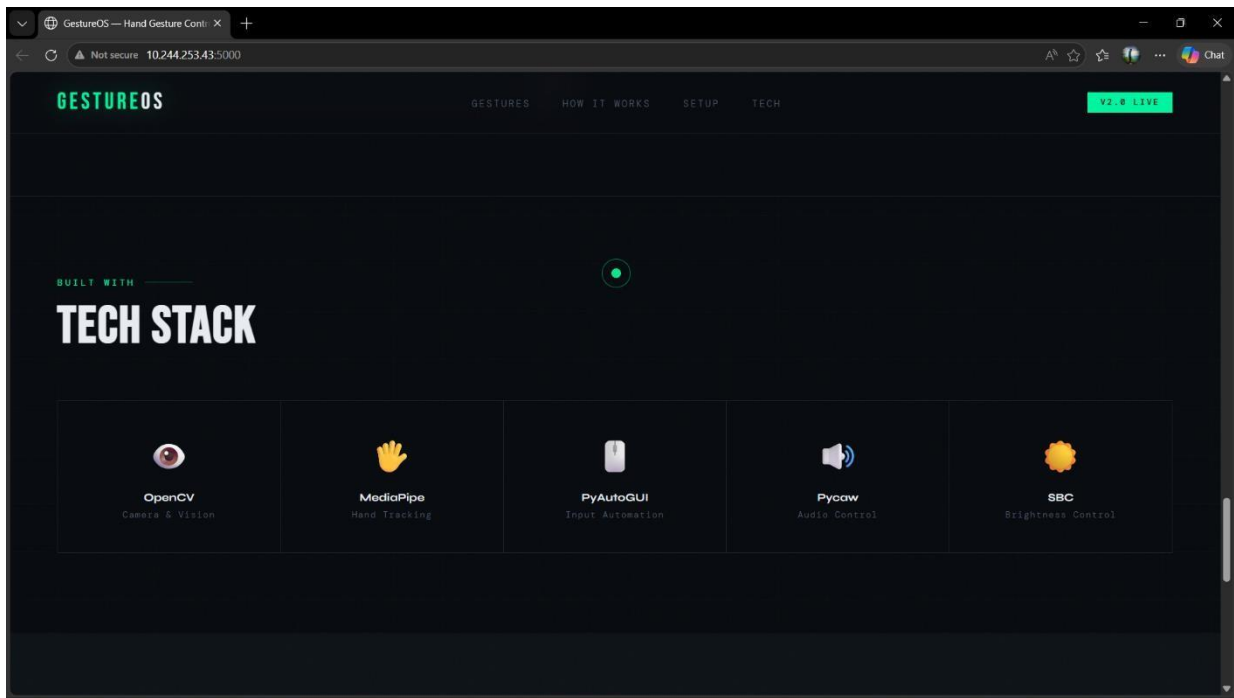


Figure 9.5: Tech Stack of the Proposed System

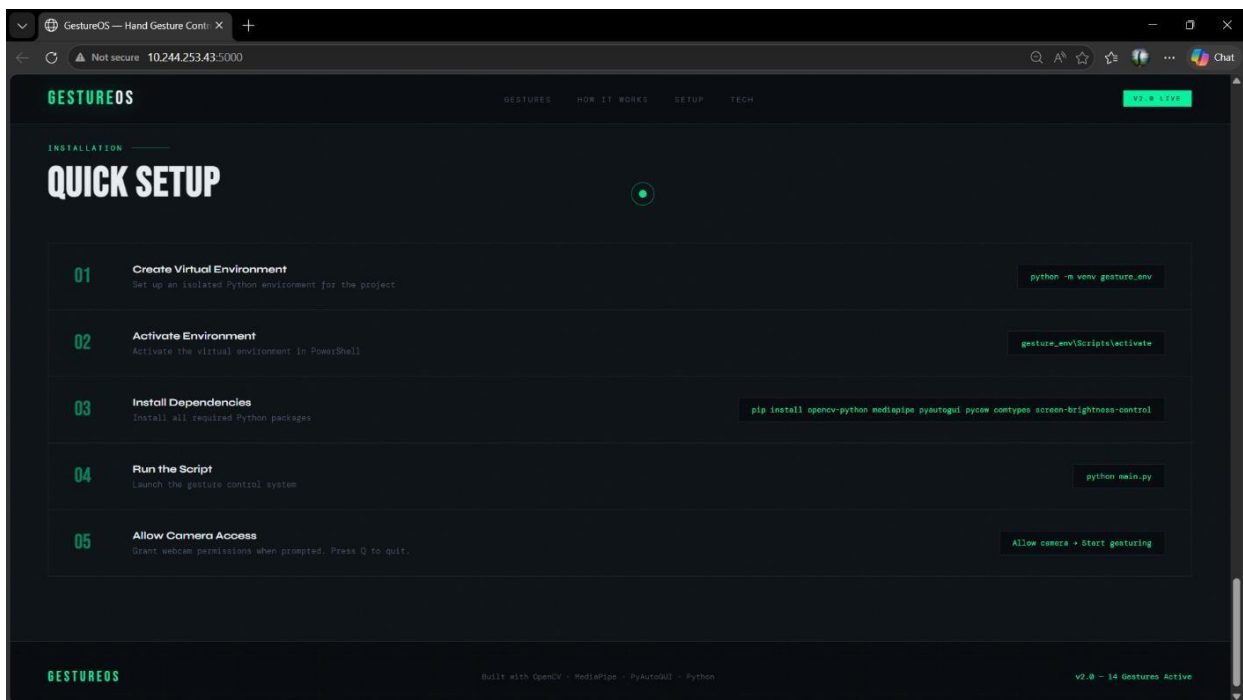


Figure 9.6: Quick setup page for the Proposed System

REFERENCES

- [1] 1R. Bharath, 2Sk. Parvez, 3M. Pavani, 4Sk. Rihana, 5K. Pavan Kumar 1Student, 2Student, 3Student, 3Student, 5Assistant Professor 1,2,3,4,5Department of Information Technology, 1,2,3,4,5Prasad V Potluri Siddhartha Institute of Technology, Vijayawada, India URL:<https://www.jetir.org/papers/JETIR2304812.pdf>
- [2] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools, 25(11), 120-126.
- [3] Dr. G. Krishna Kishore¹, M. Dhyanes², R. Vijay Kumar, MD. Ameena⁴, M. Surya Teja, R. Vijay Krishna⁶ ¹Professor, Department of Computer Science & Engineering, Dhaneekula Institute of Engineering and Technology, A.P., India URL:<https://www.ijfmr.com/papers/2024/2/14594.pdf>
- [4] Flask Documentation. Pallets Projects. Available at <https://flask.palletsprojects.com>
- [5] Flask-SocketIO Documentation. Miguel Grinberg. Available at <https://flask-socketio.readthedocs.io>
- [6] G M Trupti, Chandhan kumar, Dheeraj P, Vilas, Prasanna Kumar.S.Shivaraddi, "Virtual mouse using hand gestures," International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE), DOI: 10.17148/IJARCCE.2024.134203 URL:<https://ijarcce.com/wp-content/uploads/2024/05/IJARCCE.2024.134203.pdf>
- [7] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.L., Yong, M.G., Lee, J. and Chang, W.T. (2019). MediaPipe: A Framework for Perceiving and Processing Reality. Third Workshop on Computer Vision for AR/VR at IEEE CVPR.
- [8] MediaPipe Documentation and API Reference. Google LLC. Available at <https://developers.google.com/mediapipe>

- [9] Mitra, S. and Acharya, T. (2007). Gesture Recognition: A Survey. IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews, 37(3), 311-324.
- [10] Mr.E.Sankar, B.Nitish Bharadwaj, A.V.Vignesh Assistant Professor, Dept. Of CSE, SCSVMV (Deemed to be University) Student, Dept. Of CSE, SCSVMV (Deemed to be University)
URL:https://www.researchgate.net/publication/372165002_Virtual_Mouse_Using_Hand_Gesture
- [11] Murthy, G.R.S. and Jadon, R.S. (2009). A Review of Vision Based Hand Gestures Recognition. International Journal of Information Technology and Knowledge Management, 2(2), 405-410.
- [12] OpenCV Documentation. OpenCV Team. Available at <https://docs.opencv.org>
- [13] Pavlovic, V.I., Sharma, R. and Huang, T.S. (1997). Visual Interpretation of Hand Gestures for Human-Computer Interaction: A Review. IEEE Transactions on Pattern Analysis and Machine Intelligence, 19(7), 677-695.
- [14] PyAutoGUI Documentation. Al Sweigart. Available at <https://pyautogui.readthedocs.io>
- [15] Rautaray, S.S. and Agrawal, A. (2015). Vision Based Hand Gesture Recognition for Human Computer Interaction: A Survey. Artificial Intelligence Review, 43(1), 1-54.
- [16] Rohini Shirkar, 2Pranjali Salunke, 3Paritosh Singh, 4Sachin Shirke, 5Prof. Jagat Gaydhane Department of Information Technology, Datta Meghe College of Engineering Airoli, India URL:<https://www.ijedr.org/papers/IJEDR2304417.pdf>
- [17] screen-brightness-control Documentation. Crocket. Available at <https://screen-brightness-control.readthedocs.io>
- [18] Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.L. and Grundmann, M. (2020). MediaPipe Hands: On-device Real-time Hand Tracking. arXiv preprint arXiv:2006.10214.